

Vestisen

Guide développeur — Marketplace de mode d'occasion
Priorités · Sécurité · UX critique · Monétisation · Pièges à éviter

Backend	Spring Boot 3.2 · Java 17 · MySQL · Flyway · JWT · Stripe · Google OAuth
Frontend	Angular 17 standalone · RxJS · Chart.js · SweetAlert2
Mobile	Flutter (optionnel) · Port 9090 backend · Port 4200 frontend

1. Ordre de priorité des fonctionnalités

1 Catalogue public + fiche produit

Le visiteur doit pouvoir parcourir sans compte. C'est la vitrine — si elle ne fonctionne pas, rien d'autre ne compte.

[GET /api/annonces/public] [Filtres + recherche] [Images optimisées] [URLs SEO-friendly]

2 Authentification + profil utilisateur

Inscription rapide, vérification email, Google login. Moins de friction = plus de comptes créés.

[JWT stateless] [Google OAuth] [Email verify] [BCrypt passwords]

3 Publication annonce + messagerie

Le cœur métier. Formulaire simple, upload photos, chat acheteur/vendeur. Sans ça, pas de transactions.

[Upload multipart] [Chat polling 3s] [Modération admin]

4 Monétisation vendeur

Crédits + abonnement. Stripe d'abord, Orange Money / Wave selon la maturité de la base vendeur.

[Stripe Webhooks] [Ledger crédits] [Plans + quotas]

5 Admin + statistiques

Dashboard admin pour modérer, configurer tarifs, suivre KPIs. Peut être minimal au lancement.

[Modération annonces] [Export logs Excel] [KPIs statistiques]

2. Sécurité — règles non négociables

- Pour une marketplace, la sécurité n'est pas optionnelle. Une faille de paiement ou une usurpation de compte peut détruire la confiance définitivement.

Authentification & sessions

✓	BCrypt cost factor ≥ 12 pour les mots de passe — jamais MD5 ou SHA1
✓	JWT signé HS256 minimum, expiration courte (1h), refresh token séparé
✓	Révocation JWT au logout — table RevokedJwtToken déjà en place
✓	Rate limiting sur /api/auth/login — max 5 tentatives / IP / minute
✓	Tokens reset password à usage unique, expiration 15 min

Paielements & crédits

✓	Toujours vérifier la signature Stripe Webhook côté backend — jamais faire confiance au frontend
✓	Ledger de crédits avec CreditLedgerEntry — chaque débit/crédit tracé en base
✓	Opérations crédit idempotentes — clé idempotencyKey pour éviter les doubles crédits
■	Vérifier le solde côté backend avant chaque publication — jamais depuis le frontend
■	Gérer les transactions crédit avec verrouillage optimiste (race condition)

Upload fichiers & images

✓	Valider le type MIME côté backend (pas juste l'extension) — jpeg, png, webp uniquement
✓	Taille max stricte côté serveur — 5 Mo par image, 10 images max par annonce
✓	Renommer les fichiers avec UUID — jamais garder le nom original (path traversal)
✓	Servir les images depuis un sous-domaine ou CDN séparé — pas depuis l'API

API & données

✓	CORS strict — whitelist des origines autorisées, pas * en production
✓	Utiliser les publicId UUID dans les URLs — jamais les IDs numériques
✓	Vérifier que l'annonce appartient bien au vendeur connecté avant tout PUT/DELETE
✓	Headers de sécurité : X-Content-Type-Options, X-Frame-Options, CSP
✓	Logs d'actions admin (ActionLog) — tracer qui a fait quoi et quand

3. UX critique — friction zéro

- Sur une marketplace, l'UX est un facteur de conversion direct. Chaque friction = un utilisateur perdu.

Zone	Règle	Pourquoi
Recherche	Debounce 300ms, résultats instantanés	Pas de bouton "Rechercher" — frustrant
Favoris	Stocker en localStorage sans compte	Demander le compte à la 3e mise en favori
Images	WebP + lazy loading natif	Placeholder couleur pendant le chargement
Filtres	Synchroniser dans l'URL (queryParams)	Retour arrière = filtres restaurés
Chat	Polling 3s actif, 30s background	Éviter de surcharger le serveur inutilement

Messages rapides	"Est-ce disponible ?" en 1 clic	Réduit la friction d'initiation de contact
Publication	Max 3 étapes + sauvegarde auto	Photos → Détails → Prix, brouillon persistant
Upload photos	Drag & drop, plusieurs à la fois	La photo est l'élément #1 de décision achat
Mobile	Tester toutes les interactions au pouce	70%+ des utilisateurs viennent sur mobile
Feedback	Loader visible sur upload et paiement	Sans feedback visuel, l'utilisateur re-clique

4. Monétisation — modèle recommandé

★ Règle d'or : le vendeur doit comprendre exactement ce qu'il paie et ce qu'il obtient, avant de payer.

Structure recommandée

Plan	Prix/mois	Annonces	Crédits	Avantages
Gratuit	0 FCFA	5 max	Non inclus	Tester la plateforme
Essentiel	5 000 FCFA	20 max	Non inclus	Stats basiques, mise en avant légère
Pro	15 000 FCFA	Illimité	Non inclus	Badge pro, Top pub inclus, stats avancées
Crédits à l'unité	Selon pack	1 crédit = 1 pub	Achat à la demande	Sans engagement, vendeur occasionnel

Règles backend critiques — crédits

✓	Déduire le crédit APRES validation admin — pas à la soumission
✓	Table CreditLedgerEntry : type (PURCHASE/DEBIT/REFUND), amount, referencedId, createdAt
✓	Endpoint GET /api/credits/ledger visible du vendeur — transparence totale
✓	Crédits non remboursables clairement indiqués dans les CGU et dans l'UI
■	Verrouillage optimiste sur les transactions crédit — éviter double débit (race condition)
■	Ne jamais exposer le solde crédit dans un endpoint public — auth obligatoire

Règles backend critiques — abonnements Stripe

✓	Ne jamais activer un abonnement sans confirmation webhook Stripe customer.subscription.created
✓	Gérer les webhooks de façon idempotente — le même event peut arriver plusieurs fois
✓	Downgrade planifié : l'abonnement actif court jusqu'à fin de période, pas de coupure immédiate
✓	Email automatique J-3 avant expiration pour relancer l'abonnement
✓	Echec paiement : grace period 3 jours → suspendre annonces → notifier vendeur
■	Toujours afficher la date de prochain prélèvement et le montant dans le dashboard vendeur

5. Pièges à éviter

- Ces erreurs sont les plus courantes sur les marketplaces. Les anticiper évite de réécrire du code en production.

Erreurs backend

✗	Exposer les IDs numériques dans les URLs — utiliser publicId UUID partout
✗	Faire confiance au frontend pour le solde crédit ou le plan vendeur
✗	Oublier la vérification d'ownership — tout CRUD doit checker annonce.vendeur == userConnecte
✗	Ne pas paginer les endpoints publics — 1000 annonces sans pagination = timeout garanti
✗	Déployer avec ddl-auto=update en production — utiliser Flyway migrations
✗	Ne pas valider les types MIME des fichiers uploadés côté serveur

Erreurs frontend Angular

✗	Stocker des données sensibles dans localStorage — utiliser sessionStorage (déjà en place)
✗	Calculer le solde crédit en JS — toujours récupérer depuis l'API
✗	Oublier de désabonner les observables RxJS dans ngOnDestroy — fuite mémoire avec polling 3s
✗	Se fier uniquement aux guards Angular — le backend doit vérifier les rôles indépendamment
✗	Pas de feedback visuel sur les actions longues — toujours un loader sur upload et paiements

6. Ce qui fait la différence au lancement

Priorité	Action	Impact
Emails transactionnels	Inscription, vérif email, nouveau message, annonce approuvée	Créer vrai contact avec la marque
Qualité des images	Compression auto WebP, max 800px à l'upload	Les images floues font fuir les acheteurs
Système d'avis	Avis vendeur dès le lancement, vérifiés post-transaction	La confiance se construit sur les avis
PWA / Mobile	Notifications push pour les messages non lus	70%+ des utilisateurs sur mobile
SEO catalogue	URLs propres /produit/:publicId, balises meta dynamiques	Trafic organique gratuit dès le départ
Onboarding vendeur	Guide en 3 étapes à la première connexion	Réduire le taux d'abandon à l'inscription